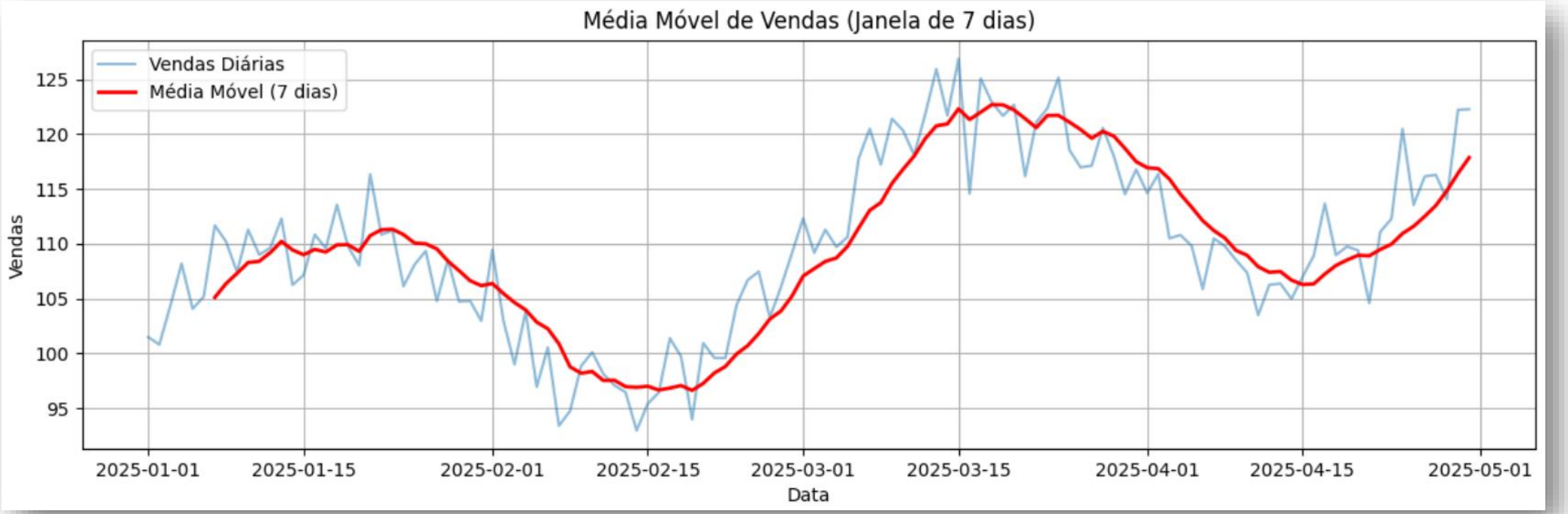
Estatísticas Dinâmicas

Daniel Freitas

Instituto J&F
Centro de Educação para os Negócios

Estatísticas Dinâmicas

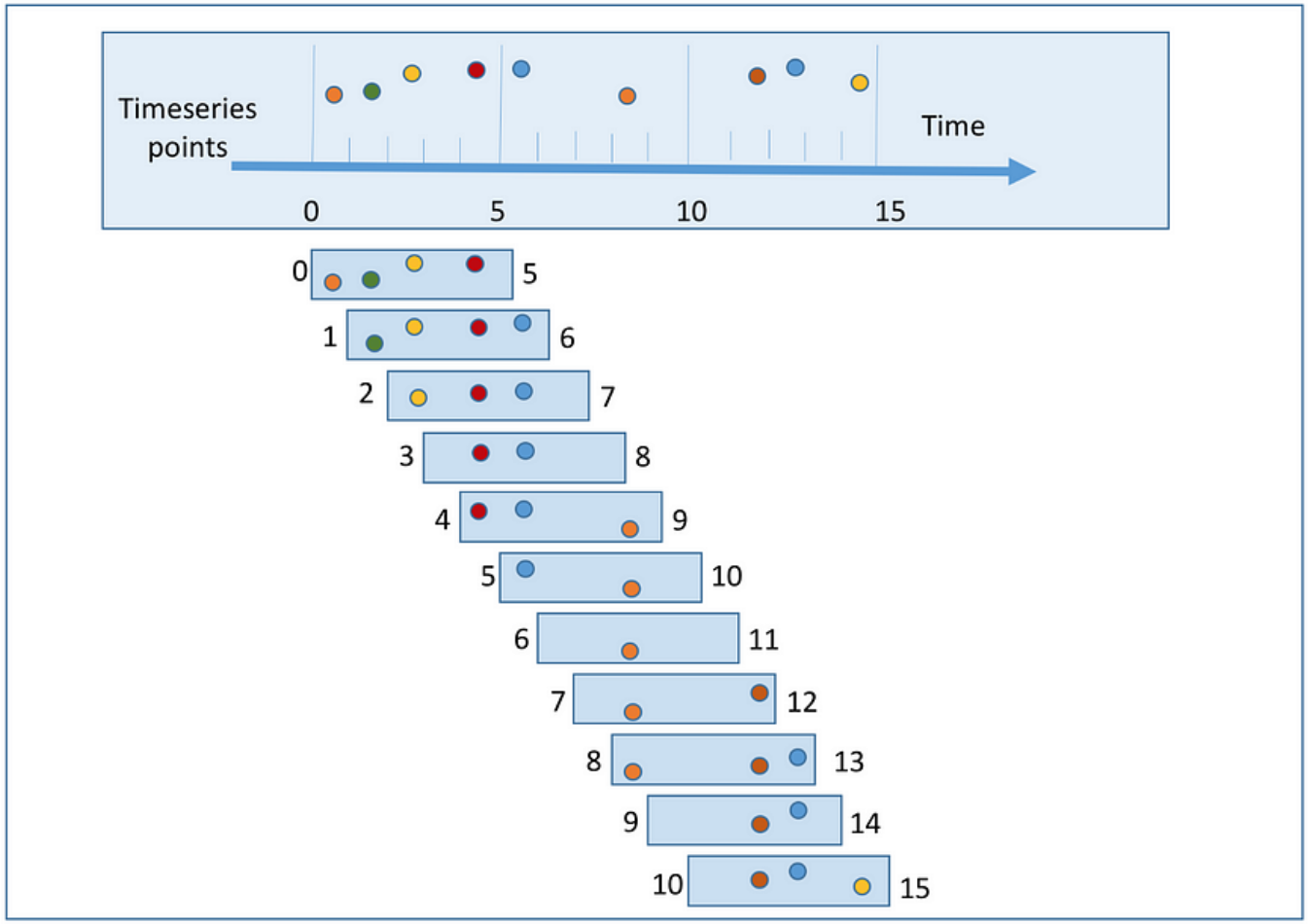
As propriedades dos dados podem variar com o tempo. Em vez de calcular uma única média ou desvio-padrão para todo o conjunto, utilizamos as **médias móveis** e **desvios-padrão móveis** para acompanhar a evolução dos parâmetros estatísticos em janelas temporais específicas.



Média Móvel

Estatísticas Dinâmicas: média móvel

A **média móvel** consiste em suavizar os dados ao longo do tempo, reduzindo o "ruído" para mostrar tendências e comportamentos subjacentes da série. Em vez de analisarmos os valores individuais, calculamos a média de um número definido de observações (chamado de janela) que "se move" ao longo da série. A média móvel simples chamamos de **SMA**



Estatísticas Dinâmicas: média móvel

Vamos calcular a média móvel simples em uma janela=5

dados = [10, 12, 15, 14, 18, 20, 17, 19, 21, 23, 22, 25, 24, 26, 28, 27, 29, 31, 30, 32]

OBS: utilize a função de serie ‘rolling()’ para criação da janela.

Estatísticas Dinâmicas: média móvel

dados = [10, 12, 15, 14, 18, 20, 17, 19, 21, 23, 22, 25, 24, 26, 28, 27, 29, 31, 30, 32]

```python
import pandas as pd
import matplotlib.pyplot as plt

# série com 20 elementos
dados = [10, 12, 15, 14, 18, 20, 17, 19, 21, 23, 22, 25, 24, 26, 28, 27, 29, 31, 30, 32]
serie = pd.Series(dados)

# média móvel com janela 5
media_movel_5 = serie.rolling(window=5).mean()

# primeiros valores (as primeiras 4 posições serão NaN)
print("Série:")
print(serie)
print("\nMédia Móvel:")
print(media_movel_5.dropna())
```

Estatísticas Dinâmicas: média móvel

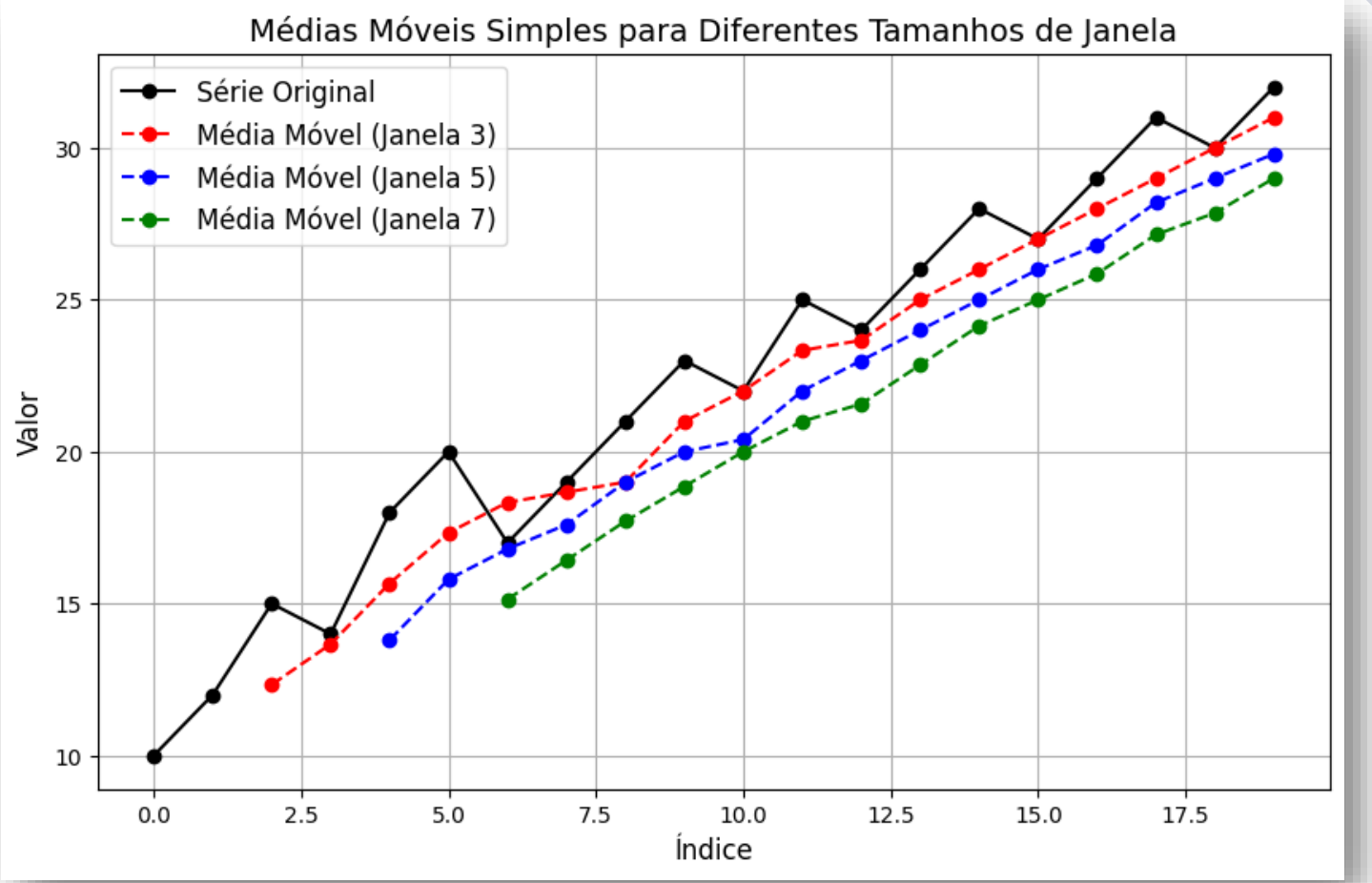
Plote a média móvel considerando janelas de 3, 5 e 7:

dados = [10, 12, 15, 14, 18, 20, 17, 19, 21, 23, 22, 25, 24, 26, 28, 27, 29, 31, 30, 32]

Estatísticas Dinâmicas: média móvel

Plote a média móvel considerando janelas de 3, 5 e 7:

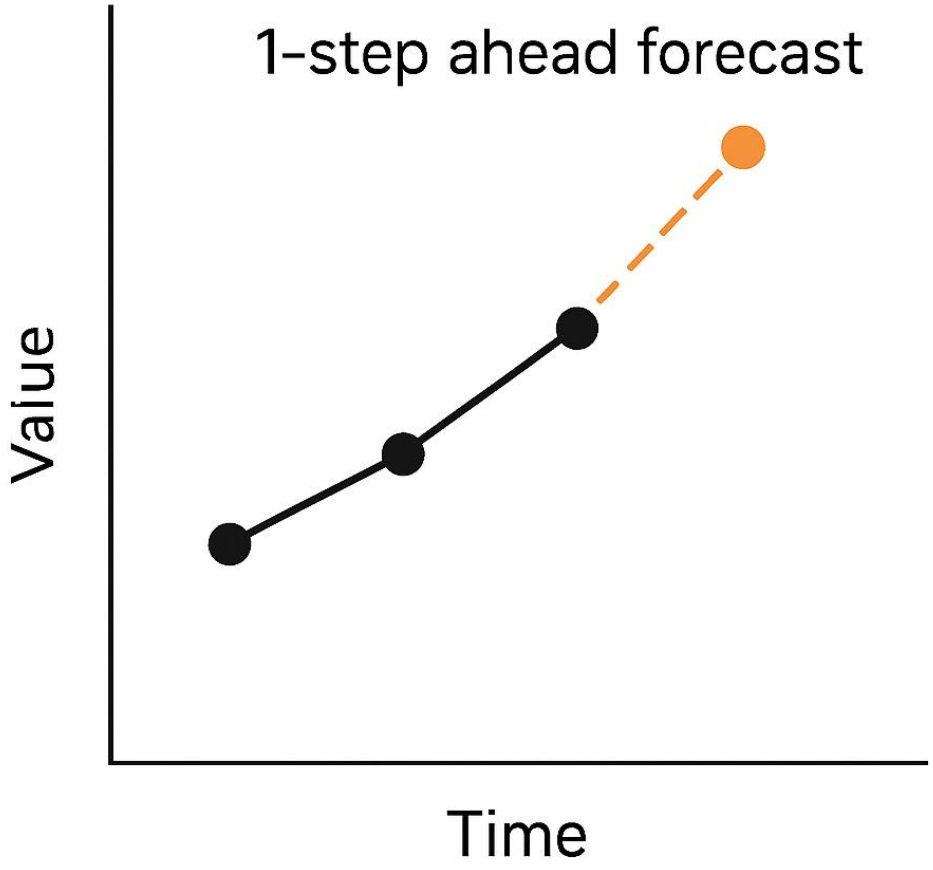
dados = [10, 12, 15, 14, 18, 20, 17, 19, 21, 23, 22, 25, 24, 26, 28, 27, 29, 31, 30, 32]



Previsão “1-step ahead”

Previsão “1-step ahead”:

A previsão ***1-step ahead*** é quando tentamos estimar qual será o próximo valor de uma sequência, utilizando apenas informações conhecidas. Ou seja, é uma previsão de curto prazo, feita passo a passo.



Base Models:

Base model é um modelo preditivo simples usado como ponto de partida para comparação com modelos mais complexos. Ele serve como referência para avaliar se modelos mais sofisticados realmente melhoram a previsão. Ao lado alguns dos principais ***base models***:

- ***Média Histórica***
- ***Média Acumulada***
- ***Média Móvel Simples (SMA)***
- ***Média Móvel Exponencial (EMA)***
- ***Taxa de Variação***
- ***Sazonal***
- ***Delta***

Erros:

$$\text{MSE} = \frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2$$

$$\text{MAE} = \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t|$$

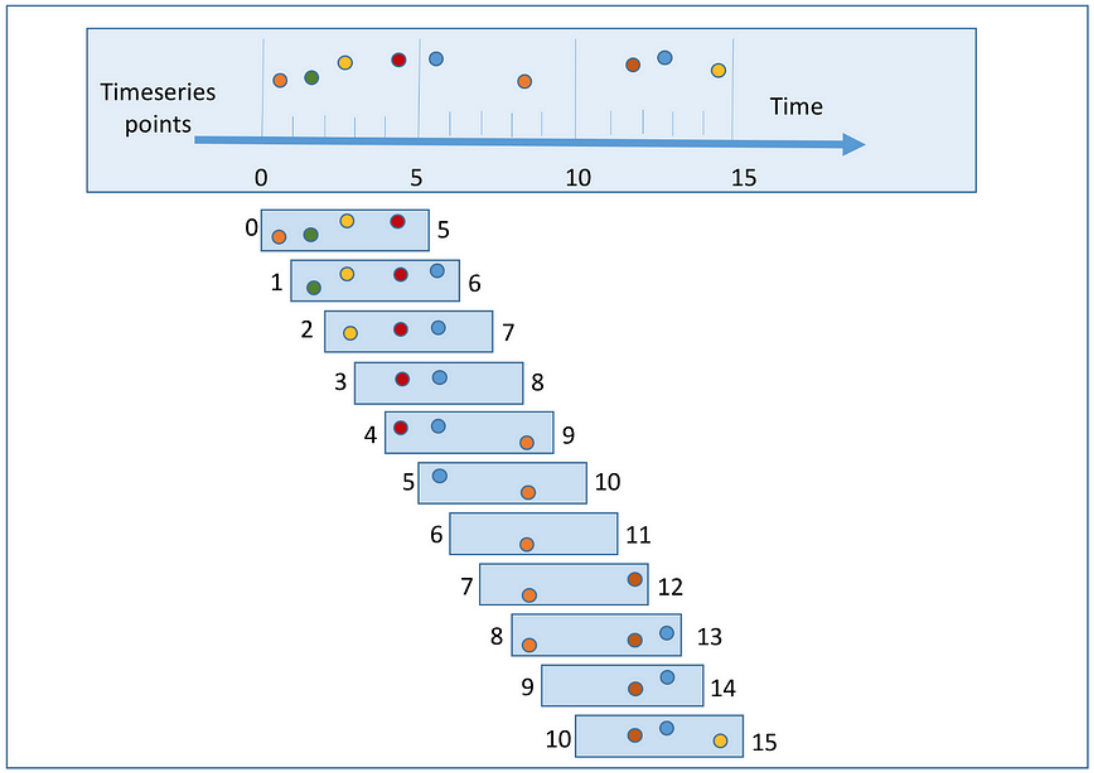
$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2}$$

$$\text{MAPE} = \frac{100}{n}\sum_{t=1}^{n}\left|\frac{y_t - \hat{y}_t}{y_t}\right| \quad (\text{desde que } y_t \neq 0)$$

Erros:

	O que Mede	Características
MAE	media da diferença absoluta entre valor real e previsto	todos os erros com o mesmo peso
MSE	media dos erros elevados ao quadrado	penaliza mais os erros grandes, sensivel a outliers
RMSE	raiz quadrada do MSE, erro na mesma escala dos dados	parecida com o MAE, mas mais severa com erros grandes
MAPE	erro medio expresso em porcentagem	bom pra comparar entre escalas, mas problemático com valores pequenos

Base model: média móvel

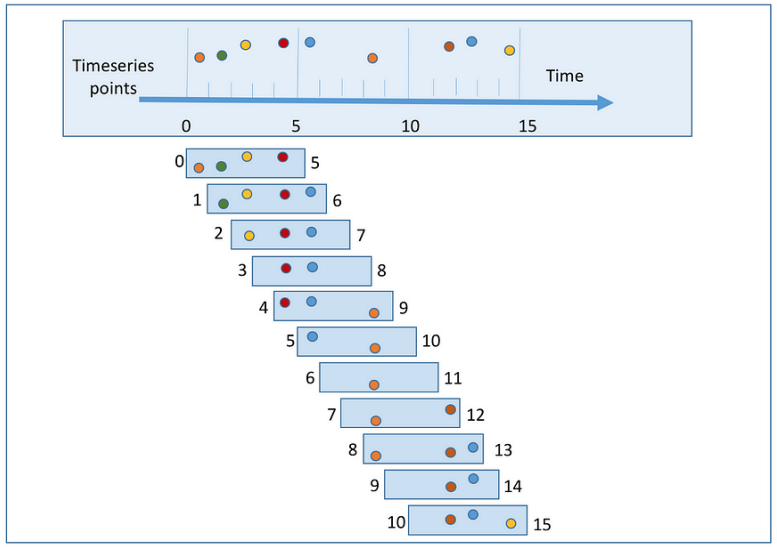


$$\hat{y}_{\text{próximo}} = \frac{1}{k}\sum_{i=0}^{k-1} y_{t-i}$$

Prevê o próximo valor de uma série utilizando a média dos últimos k pontos observados

Base model: média móvel

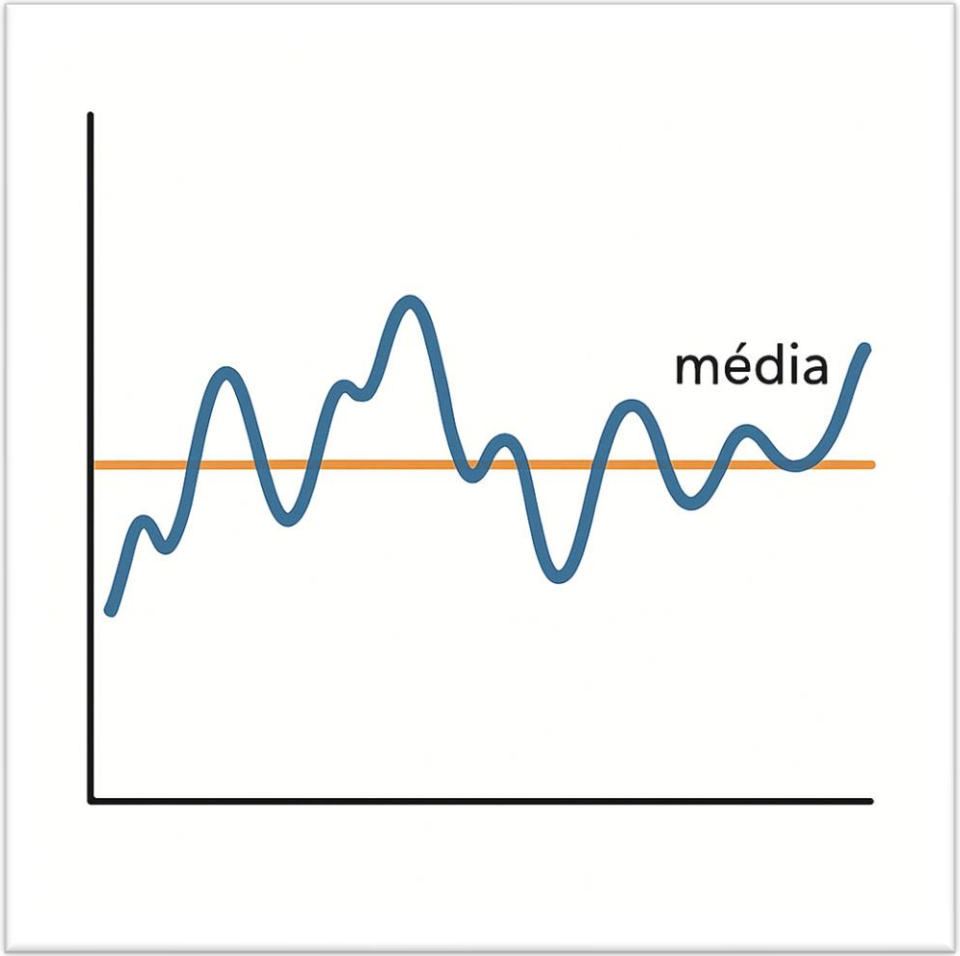
$$\hat{y}_{\text{próximo}} = \frac{1}{k}\sum_{i=0}^{k-1} y_{t-i}$$




```python
# capturando tamanho da serie
tamanho_serie = len(serie)

# para relatorio detalhado com erro
sma_detalhado = pd.DataFrame({})
sma_detalhado['Dados'] = serie

# df para resultados
resultado = pd.DataFrame({},columns=['Janela','MAE'])

# variando as janels
for janela in range(1,tamanho_serie):

    # calculando as medias moveis
    medias_moveis = serie.rolling(window = janela).mean().shift(1)

    # medias moveis e os desvios gerados
    sma_detalhado[f'sma_{janela}'] = medias_moveis
    sma_detalhado[f'erro_{janela}'] = np.abs(medias_moveis-serie)

    # dropando os duplicados para o calculo do erro
    medias_moveis = medias_moveis.dropna()
    serie_adaptada = serie[medias_moveis.index]

    # calculo do mae
    erro = mean_absolute_error(medias_moveis,serie_adaptada)

    nova_linha = pd.DataFrame({'Janela':[janela],
                               'MAE': [erro]})
    resultado = pd.concat([resultado, nova_linha], ignore_index=True)
```

Base Model: média histórica

Base model: média histórica



$$\hat{y}_{\text{próximo}} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

Prevê o próximo valor de uma série utilizando a média global dos dados

Base model: média móvel global

$$[10,\ 12,\ 14,\ 13,\ 15]$$

Base model: média móvel global

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Calcular média

Base model: média móvel global

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Calcular média $\mu = 12.8$
2. Medir erros

Base model: média móvel global

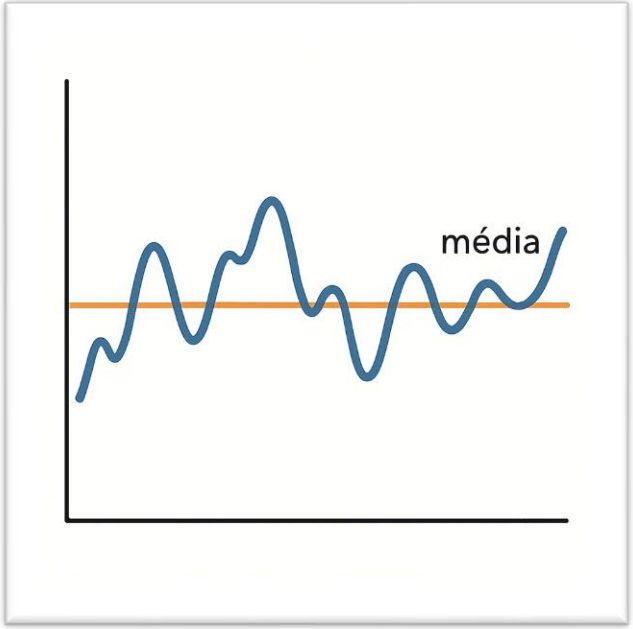
$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Calcular média $\mu = 12.8$
2. Medir erros

[NAN, 0.8, 1.2, 0.2, 2.2]

Base model: média histórica

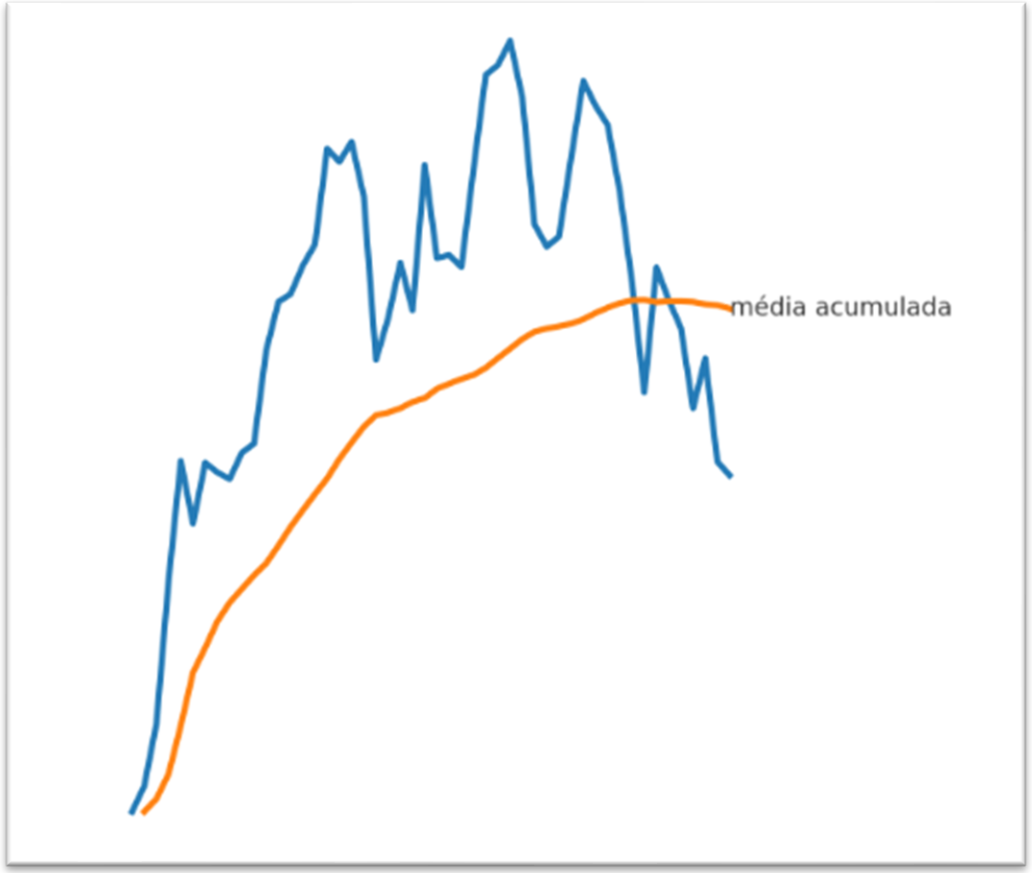
$$\hat{y}_{\text{próximo}} = \frac{1}{n}\sum_{i=1}^{n} y_i$$




```python
# calcula a média de toda a série
media_global = serie.mean()

# cria a previsão com a medi historica,
# --> usando shift(1) para simular 1 passo a frente
previsao = pd.Series(media_global, index=serie.index).shift(1)

# calcula o erro absoluto (apenas para o relatorio de erros)
erro = np.abs(previsao - serie)

# adiciona ao dataframe detalhado
detalhado['erro'] = erro

# remove valores nulos para o cálculo do erro
# funcao nao aceita NAN
previsao = previsao.dropna()
serie_alinhada = serie[previsao.index]

mae = mean_absolute_error(previsao, serie_alinhada)

resultado = pd.DataFrame({'Modelo': ['Media Historica'], 'MAE': [mae]})
```

Base Model: média acumulada

Base model: média acumulada



$$\widehat{x}_t = \frac{1}{t-1}\sum_{i=0}^{t-2} x_i \quad \text{para } t \geq 2$$

Prevê cada novo valor com a média de todos os valores anteriores, adaptando-se ao longo do tempo com mais dados observados

Base model: média acumulada

$$[10,\ 12,\ 14,\ 13,\ 15]$$

Base model: média acumulada

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Calcular média apenas dos pontos anteriores da série

Base model: média acumulada

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Calcular média apenas dos pontos anteriores da série

[NAN, 10, 11, 12, 12.25]

Base model: média acumulada

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Calcular média apenas dos pontos anteriores da série

[NAN, 10, 11, 12, 12.25]

2. Calcular erros

Base model: média acumulada

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Calcular média apenas dos pontos anteriores da série

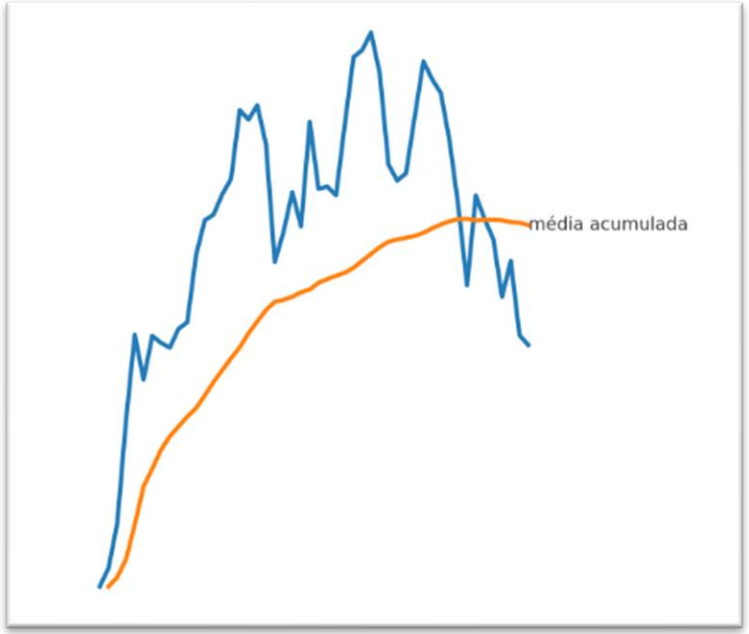
[NAN, 10, 11, 12, 12.25]

2. Calcular erros

[NAN, 2, 3, 1, 2.75]

Base model: média acumulada

$$\widehat{x}_t = \frac{1}{t-1}\sum_{i=0}^{t-2} x_i \quad \text{para } t \geq 2$$

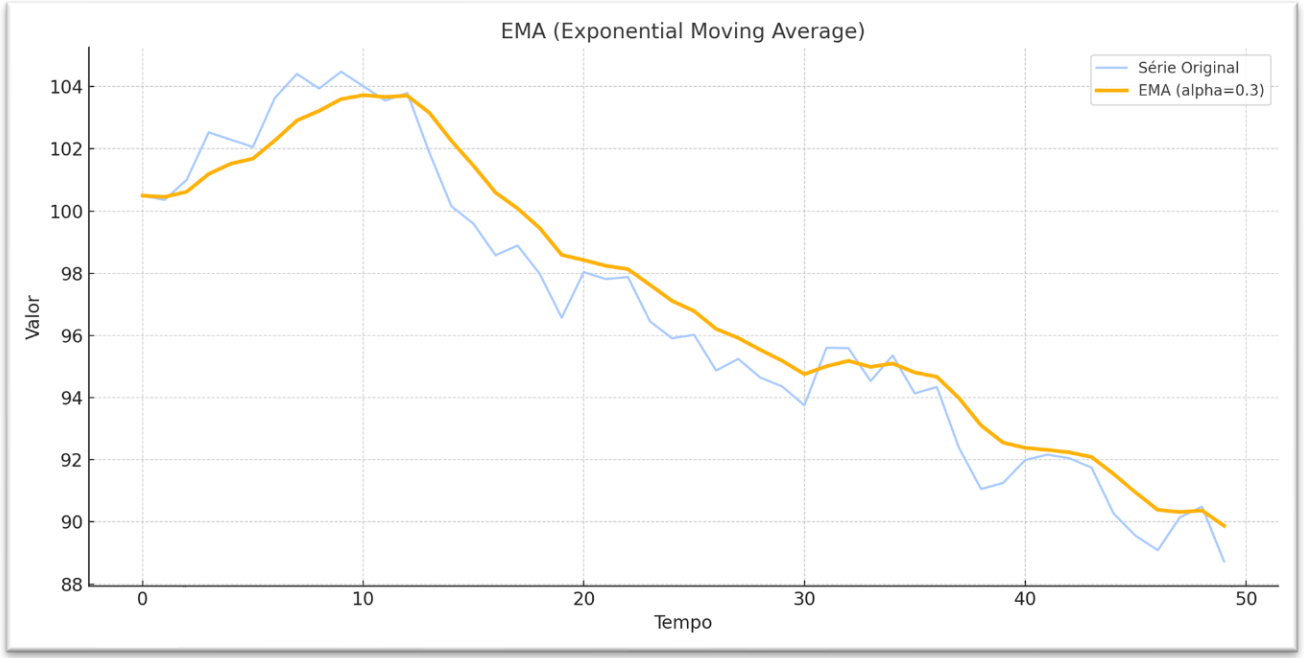



```python
# calcula a média acumulada até t-1
previsao = []
for i in range(len(serie)):
    if i == 0:
        previsao.append(np.nan)  # não há previsão no primeiro valor
    else:
        previsao.append(serie.iloc[:i].mean())

previsao = pd.Series(previsao, index=serie.index)

# ero absoluto
erro = np.abs(previsao - serie)
detalhado['erro'] = erro

# remove nulos para calcular o erro
previsao = previsao.dropna()
serie_alinhada = serie[previsao.index]
mae = mean_absolute_error(previsao, serie_alinhada)
```

Base Model: média móvel exponencial

Base model: média móvel exponencial (EMA)

$$\mathrm{EMA}_t = \alpha \cdot x_{t-1} + (1-\alpha) \cdot \mathrm{EMA}_{t-1}$$



Média que atribui **pesos maiores aos valores mais recentes da série**, decaindo exponencialmente para os mais antigos

EMA_t Previsão para o tempo t

EMA_{t-1} EMA no tempo anterior

1 atual

0 histórico

α Peso dado ao valor mais recente

x_{t-1} Valor observado no tempo anterior (t – 1)

Base model: média móvel exponencial (EMA)

Vamos calcular a EMA para alpha = 0.8 para a serie abaixo:

$$[10,\ 12,\ 14,\ 13,\ 15]$$

Base model: média móvel exponencial (EMA)

Vamos calcular a EMA para alpha = 0.8 para a serie abaixo:

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Não se atribui valor ao primeiro da série, e considera-se o valor anterior como previsão para o segundo;

Base model: média móvel exponencial (EMA)

Vamos calcular a EMA para alpha = 0.8 para a serie abaixo:

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Não se atribui valor ao primeiro da série, e considera-se o valor anterior como previsão para o segundo;

[NAN, 10, , ,]

Base model: média móvel exponencial (EMA)

Vamos calcular a EMA para alpha = 0.8 para a serie abaixo:

$$[10,\ 12,\ 14,\ 13,\ 15]$$

$$[\text{NAN},\ 10,\ ,\ ,\ ]$$

1. Não se atribui valor ao primeiro da série, e considera-se o valor anterior como previsão para o segundo;

2. Considera-se **80% do valor atual** e **20% da previsão anterior**.

0.8 x 12 + **0.2 x 10** = 11.6

Base model: média móvel exponencial (EMA)

Vamos calcular a EMA para alpha = 0.8 para a serie abaixo:

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Não se atribui valor ao primeiro da série, e considera-se o valor anterior como previsão para o segundo;

[NAN, 10, 11.6, ,]

2. Considera-se **80% do valor atual** e **20% da previsão anterior**.

0.8 x 12 + **0.2 x 10** = 11.6

Base model: média móvel exponencial (EMA)

Vamos calcular a EMA para alpha = 0.8 para a serie abaixo:

$$[10,\ 12,\ 14,\ 13,\ 15]$$

3. Calcula-se para toda a série:

Base model: média móvel exponencial (EMA)

Vamos calcular a EMA para alpha = 0.8 para a serie abaixo:

$$[10,\ 12,\ 14,\ 13,\ 15]$$

3. Calcula-se para toda a série:

[NAN, 10, 11.6, 13.52, 13.104]

Base model: média móvel exponencial (EMA)

Vamos calcular a EMA para alpha = 0.8 para a serie abaixo:

$$[10,\ 12,\ 14,\ 13,\ 15]$$

3. Calcula-se para toda a série:

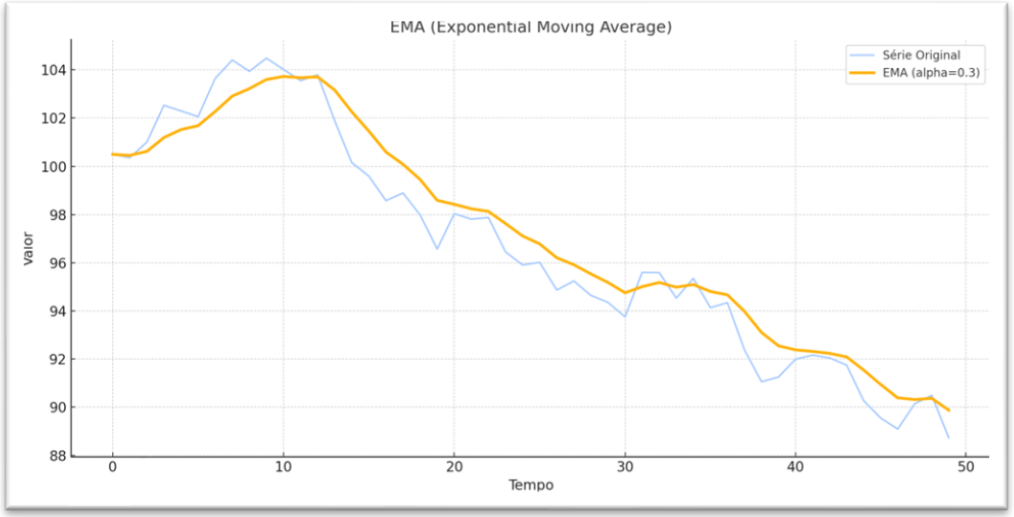
[NAN, 10, 11.6, 13.52, 13.104]

4. Calcula-se o erro:

[NAN, 2, 2.4, 0.52, 1.896]

Base model: média móvel exponencial (EMA)

$$\mathrm{EMA}_t = \alpha \cdot x_{t-1} + (1-\alpha) \cdot \mathrm{EMA}_{t-1}$$

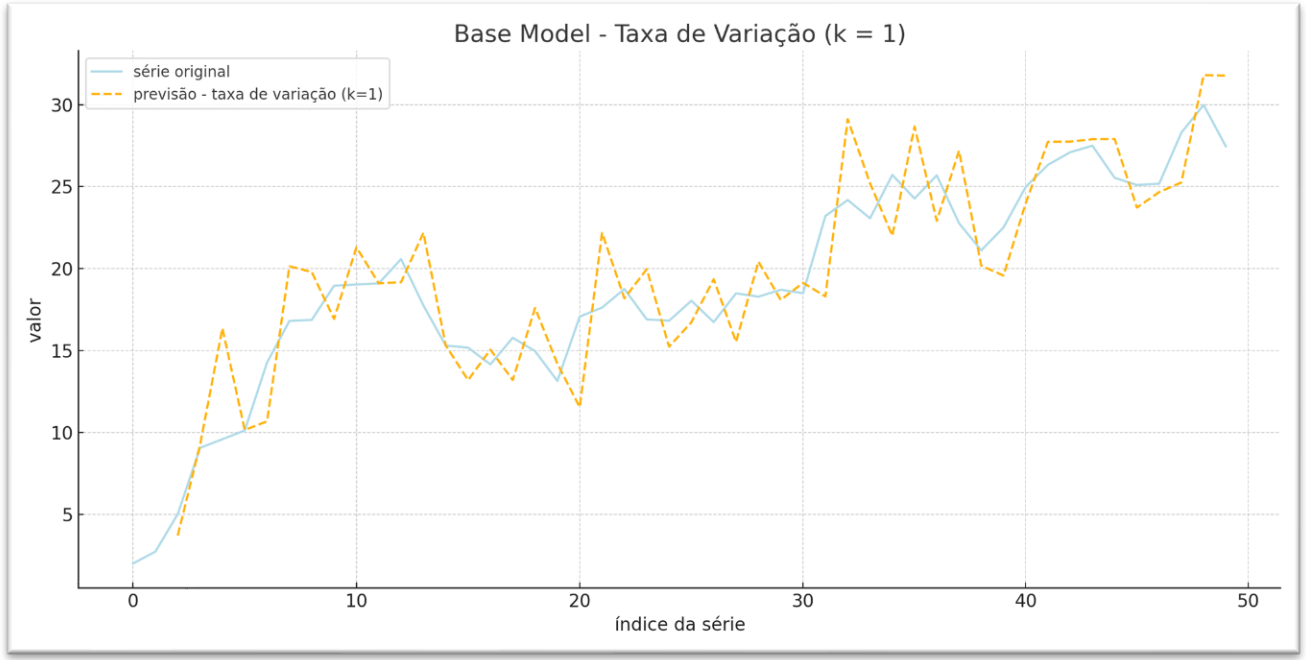



```python
# variando os valores de alpha entre 0.1 e 0.9 com passo de 0.1
alphas = np.round(np.arange(0.1, 1.0, 0.1), 2)

# para cada valor de alpha, calcular o EWM (EMA)
for alpha in alphas:

    # calculando a mme com shift para prever o valor seguinte
    mme = serie.ewm(alpha=alpha, adjust=False).mean().shift(1)

    # registrando a mme e erro no df detalhado
    ema_detalhado[f'mme_{alpha}'] = mme
    ema_detalhado[f'erro_{alpha}'] = np.abs(mme - serie)

    # removendo valores nulos
    mme = mme.dropna()
    serie_adaptada = serie[mme.index]

    # calculando o mae
    erro = mean_absolute_error(mme, serie_adaptada)

    nova_linha = pd.DataFrame({'Alpha': [alpha],
                               'MAE': [erro]})
    resultado = pd.concat([resultado, nova_linha], ignore_index=True)
```

Base Model: taxa de variação

Base model: taxa de variação

$$\hat{x}_{t+1} = x_t \cdot \left(1 + \frac{x_t - x_{t-k}}{x_{t-k}}\right)$$



Prevê cada novo valor com base na variação percentual entre o valor atual e o valor de k **períodos** atrás, projetando essa mesma taxa no próximo ponto.

Base model: taxa de variação

Vamos calcular a taxa de variação como exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

Base model: taxa de variação

Vamos calcular a taxa de variação como exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Não atribuímos valores aos primeiros k+1 valores da serie:

[NAN, NAN, NAN, ,]

Base model: taxa de variação

Vamos calcular a taxa de variação como exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Não atribuímos valores aos primeiros k+1 valores da serie:

[NAN, NAN, NAN, ,]

2. Calculamos a taxa de variação entre o **x_t-k** e o **x_t**:

Base model: taxa de variação

Vamos calcular a taxa de variação como exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Não atribuímos valores aos primeiros k+1 valores da serie:

[NAN, NAN, NAN, ,]

2. Calculamos a taxa de variação entre o **x_t-k** e o **x_t**:

$$taxa = \frac{14 - 10}{10} = \frac{4}{10} = 0.4$$

Base model: taxa de variação

Vamos calcular a taxa de variação como exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

3. Aplicamos a taxa ao **x_t** para a próxima previsão:

$$previsão = \mathbf{14} * (1 + taxa)$$

$$previsão = \mathbf{14} * (1 + 0.4) = 19.6$$

Base model: taxa de variação

Vamos calcular a taxa de variação como exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

3. Aplicamos a taxa ao x_t para a próxima previsão.

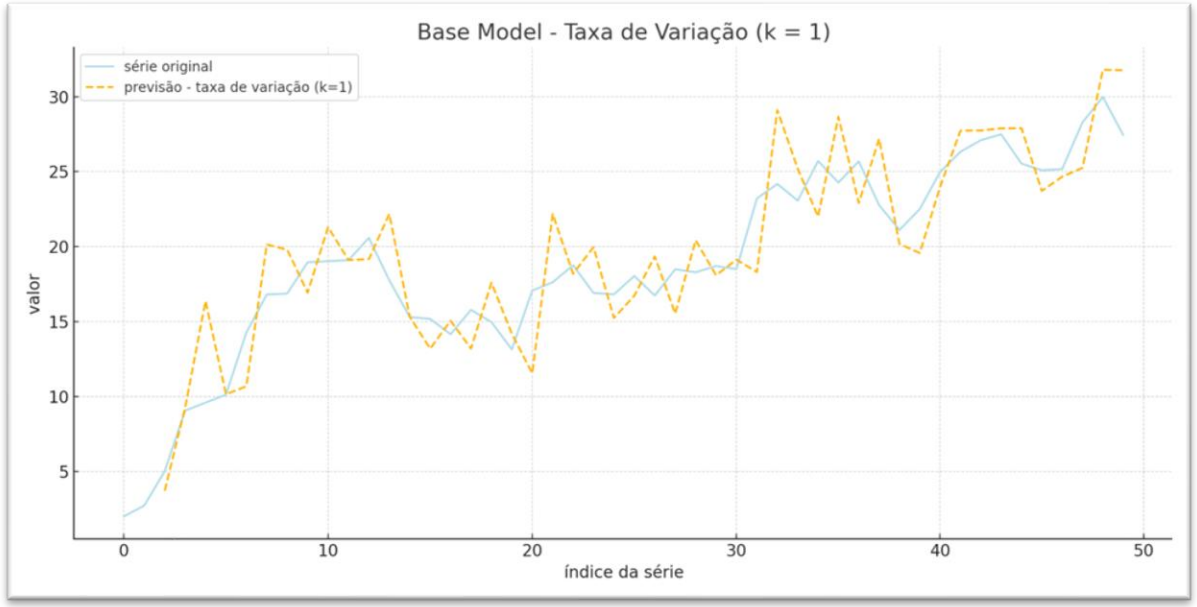
4. Aplicamos a toda a série e calculamos o erro.

[NAN, NAN, NAN, 19.6, 14.083]

[NAN, NAN, NAN, 6.6, 0.9166]

Base model: taxa de variação

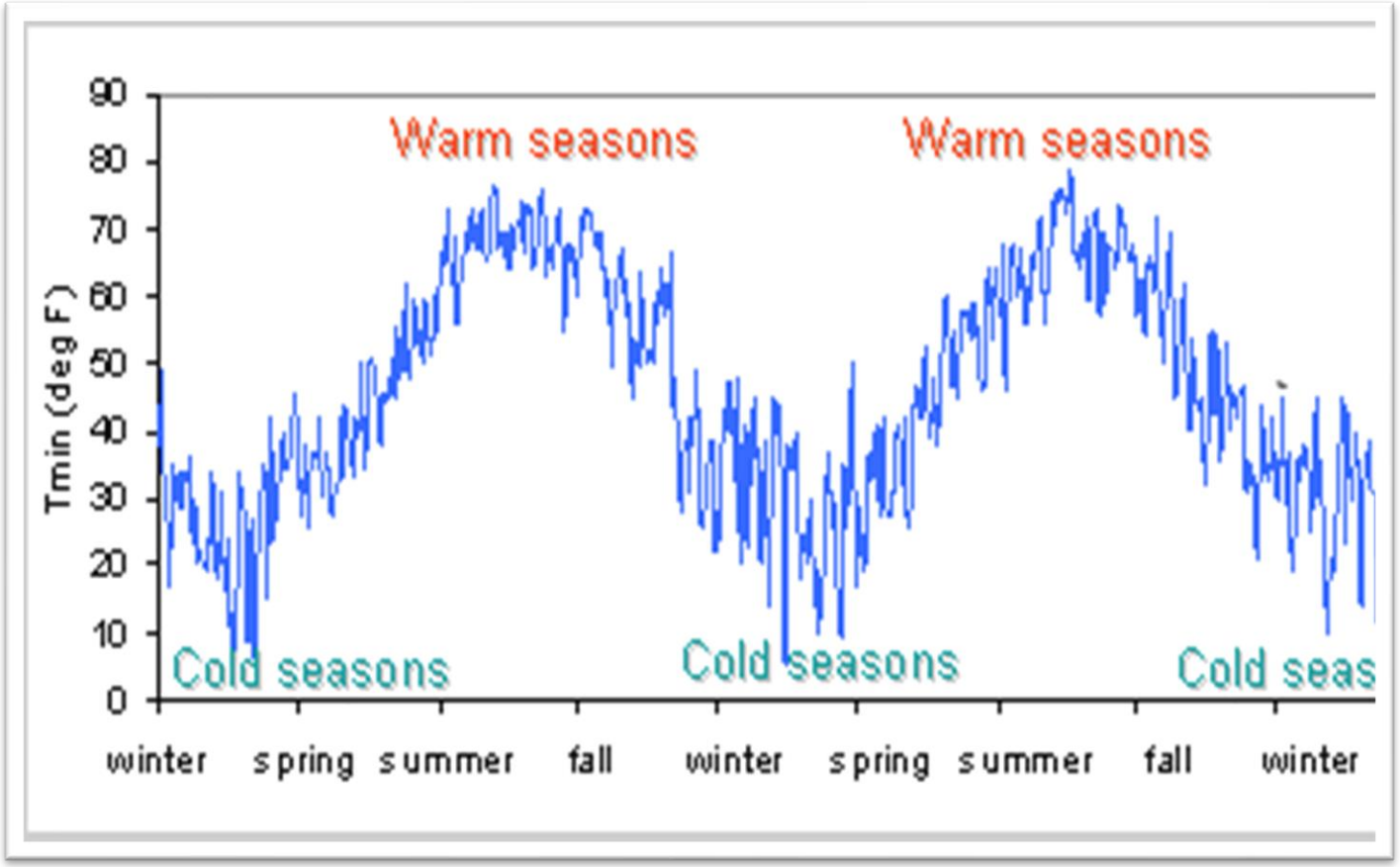
$$\hat{x}_{t+1} = x_t \cdot \left(1 + \frac{x_t - x_{t-k}}{x_{t-k}}\right)$$




```python
# testando diferentes janelas - atras
for janela in range(1, tamanho_serie - 1):

    previsoes = []
    indices = []

    # p/ cada valor -> calcula a taxa de variacao com base na janela atual
    # usa essa taxa para prever o prox ponto da serie
    for i in range(janela, tamanho_serie - 1):
        valor_atual = serie[i]
        valor_passado = serie[i - janela]

        # evita divisao por zero
        if valor_passado == 0:
            continue

        # calcula a taxa de variacao
        taxa = (valor_atual - valor_passado) / valor_passado

        # aplica a taxa ao valor atual
        previsao = valor_atual * (1 + taxa)

        previsoes.append(previsao)
        indices.append(i + 1)  # prevendo o próximo ponto
```

Base Model: seasonal naive

Base model: seasonal naive



$$\hat{x}_t = x_{t-s}$$

Realiza a previsão do próximo valor da série com base no valor observado a ***s*** períodos atrás. Assume-se uma repetição sazonal.

Base model: seasonal naive

Vamos calcular a taxa de variação como exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

Base model: seasonal naive

Vamos calcular a taxa de variação como exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Não atribuímos valores aos primeiros k valores da serie:

[NAN, NAN, , ,]

Base model: seasonal naive

Vamos calcular a taxa de variação como exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Não atribuímos valores aos primeiros k valores da serie:

[NAN, NAN, , ,]

2. “Copiamos” o valor da posição t-k da série como previsão e calculamos o erro:

[NAN, NAN, 10, 12, 14]

Base model: seasonal naive

Vamos calcular a taxa de variação como exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Não atribuímos valores aos primeiros k valores da serie:

[NAN, NAN, , ,]

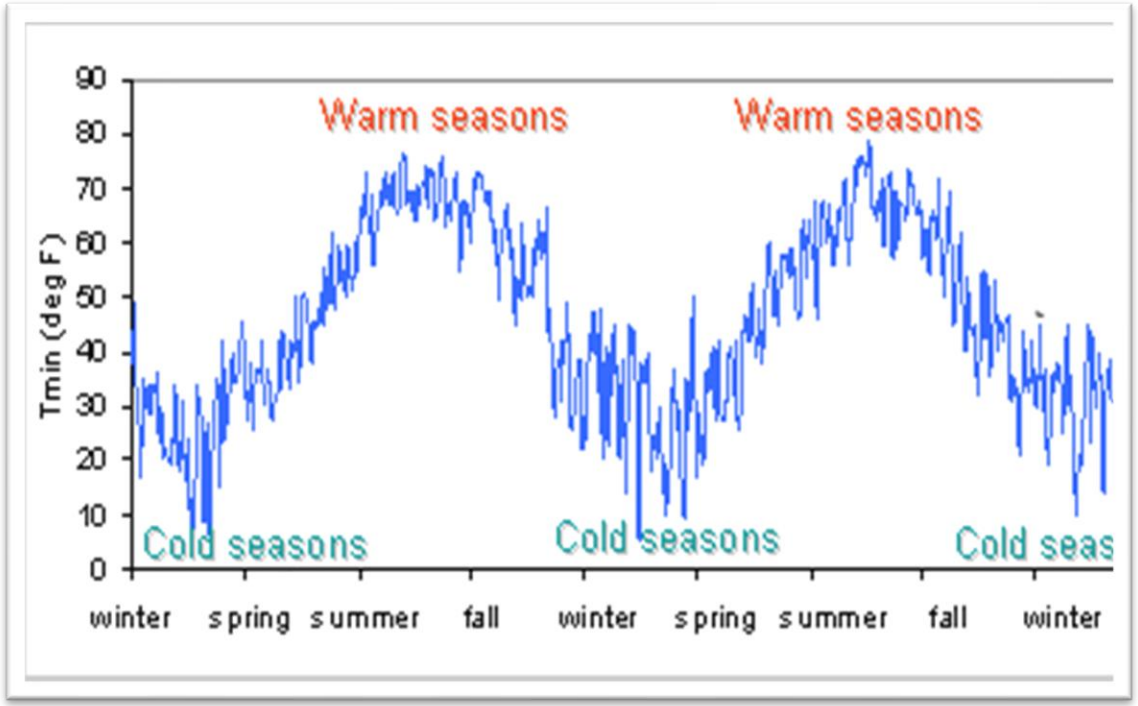
2. “Copiamos” o valor da posição t-k da série como previsão e calculamos o erro:

[NAN, NAN, 10, 12, 14]

[NAN, NAN, NAN, 6.6, 0.9166]

Base model: seasonal naive

$$\hat{x}_t = x_{t-s}$$

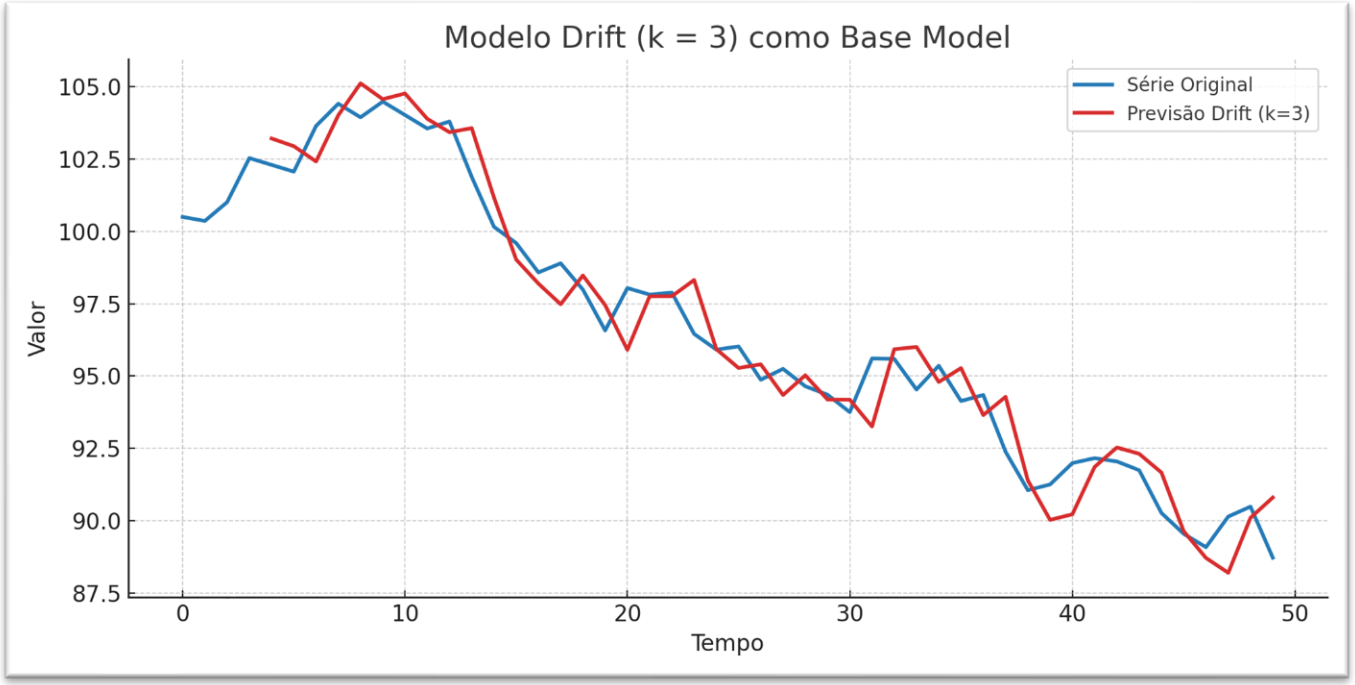



```python
# testa varias sazonalidades
for s in range(1, tamanho_serie // 2):

    # cria a previsao usando o valor de s atras
    previsao = serie.shift(s)

    # pega os valores reais pra comparar
    real_serie = serie[previsao.index]

    # salva as previsoes e erros no detalhado
    sazonal_detalhado[f'szn_{s}'] = previsao
    sazonal_detalhado[f'erro_{s}'] = np.abs(previsao - real_serie)

    # calcula o erro medio se tiver previsao
    previsao = previsao.dropna()
    real_serie = real_serie[previsao.index]

    if len(previsao) > 0:
        erro = mean_absolute_error(previsao, real_serie)
        nova_linha = pd.DataFrame({'Sazonalidade': [s], 'MAE': [erro]})
        resultado = pd.concat([resultado, nova_linha], ignore_index=True)
```

Base Model: Delta

Base model: Delta (drift)

$$\hat{x}_{t+1} = x_t + \frac{x_t - x_{t-k}}{k}$$



Projeta o próximo valor com base na **tendência média dos últimos k períodos**, assumindo que essa variação continuará no mesmo ritmo

Base model: Delta

Vamos calcular a previsão com delta com o exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

Base model: Delta

Vamos calcular a previsão com delta com o exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Não atribuímos valores aos primeiros k+1 valores da serie:

[NAN, NAN, NAN, ,]

Base model: Delta

Vamos calcular a previsão com delta com o exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

1. Não atribuímos valores aos primeiros k+1 valores da serie:

[NAN, NAN, NAN, ,]

2. Calculamos a diferença entre o **x_t-k** e o **x_t** e dividimos pela janela (k):

$$delta = \frac{14 - 10}{2} = 2$$

Base model: Delta

Vamos calcular a previsão com delta com o exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

3. Adicionamos o delta ao **x_t** para a próxima previsão:

$$previsão = \mathbf{14} + delta$$

$$previsão = \mathbf{14} + 2 = 16$$

Base model: Delta

Vamos calcular a previsão com delta com o exemplo para k = 2 (janela):

$$[10,\ 12,\ 14,\ 13,\ 15]$$

3. Adicionamos o delta ao x_t para a próxima previsão:

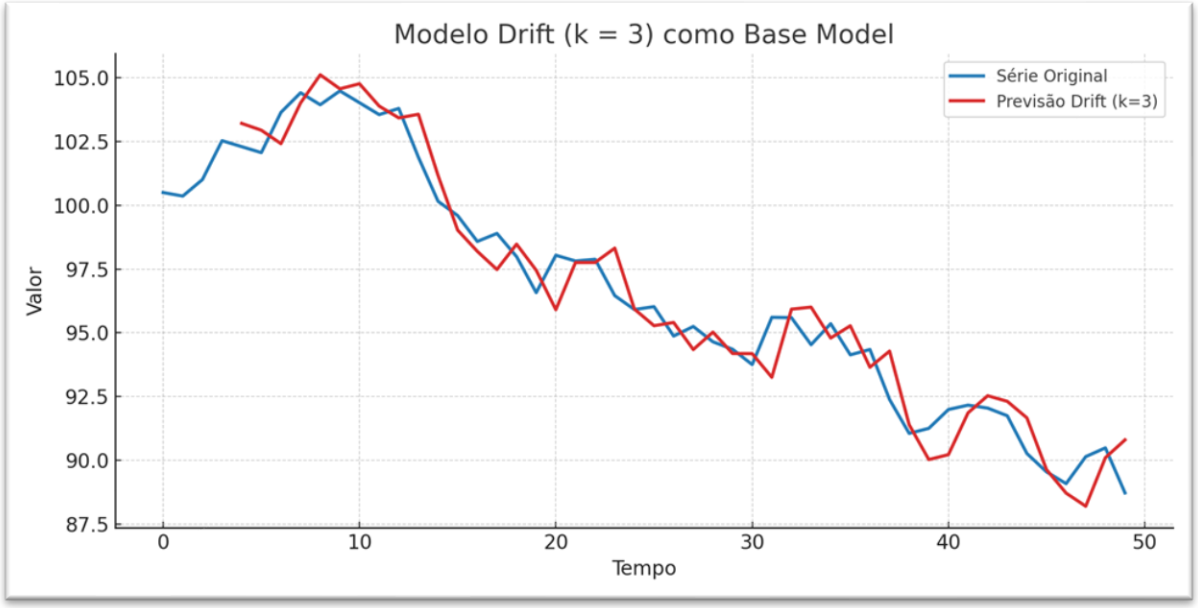
4. Aplicamos a toda a série e calculamos o erro.

[NAN, NAN, NAN, 16, 13.5]

[NAN, NAN, NAN, 3, 1.5]

Base model: Delta (drift)

$$\hat{x}_{t+1} = x_t + \frac{x_t - x_{t-k}}{k}$$




```python
# testa varias janelas -> aq sao os K valores
for janela in range(1, tamanho_serie - 1):

    previsoes = []  # lista pra guardar as previsoes
    indices = []    # indices das previsoes

    # anda pela serie pra calcular o delta
    for i in range(janela, tamanho_serie - 1):
        valor_atual = serie[i]
        valor_passado = serie[i - janela]
        delta = (valor_atual - valor_passado) / janela
        previsao = valor_atual + delta

        previsoes.append(previsao)
        indices.append(i + 1)  # previsao pro proximo ponto

    # cria uma serie com as previsoes
    previsao_serie = pd.Series(previsoes, index=indices)

    # pega os valores reais pra comparar
    real_serie = serie[previsao_serie.index]

    # salva as previsoes e erros no detalhado
    delta_detalhado[f'delta_{janela}'] = previsao_serie
    delta_detalhado[f'erro_{janela}'] = np.abs(previsao_serie - real_serie)
```

Exercícios

Exercício 1: análise de séries

Encontre o melhor base model para as séries mensais a seguir ao longo de um ano para a previsão:

1. Vendas de roupas de frio - [120, 135, 128, 122, 119, 132, 126, 121, 118, 134, 127, 120]

2. Acesso diários ao app, pós campanha - [12, 15, 22, 28, 35, 36, 36, 36, 36, 36, 36, 36]

3. n. de itens médios em uma loja - [3.1, 3.0, 3.2, 3.0, 3.3, 3.1, 3.2, 3.2, 3.1, 3.0, 3.2, 3.1]

4. Precificação de um produto - [100.0, 102.0, 104.0, 106.0, 108.0, 110.0, 112.0, 114.0]

5. Reclamações em um app - [80, 75, 77, 76, 78, 74, 73, 76, 77, 79, 78, 75]

Instituto J&F

Centro de Educação para os Negócios